Neural Network for Handwritten Digit Classification (MNIST)

Experiment: Neural Networks with TensorFlow/Keras

In [1]:
```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import numpy as np
import matplotlib.pyplot as plt

print("TensorFlow version:", tf.__version__)
```

```
WARNING: All log messages before absl::InitializeLog() is called are writt
en to STDERR
I0000 00:00:1773384458.585997   18160 port.cc:153] oneDNN custom operation
s are on. You may see slightly different numerical results due to floating
-point round-off errors from different computation orders. To turn them of
f, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
I0000 00:00:1773384458.586584   18160 cudart_stub.cc:31] Could not find cu
da drivers on your machine, GPU will not be used.
I0000 00:00:1773384458.652547   18160 cpu_feature_guard.cc:227] This Tenso
rFlow binary is optimized to use available CPU instructions in performance
-critical operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operatio
ns, rebuild TensorFlow with the appropriate compiler flags.
WARNING: All log messages before absl::InitializeLog() is called are writt
en to STDERR
I0000 00:00:1773384460.345195   18160 port.cc:153] oneDNN custom operation
s are on. You may see slightly different numerical results due to floating
-point round-off errors from different computation orders. To turn them of
f, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
I0000 00:00:1773384460.345565   18160 cudart_stub.cc:31] Could not find cu
da drivers on your machine, GPU will not be used.
TensorFlow version: 2.21.0
```

Load MNIST Dataset

In [3]:
```
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()

print("Training samples:", x_train.shape)
print("Test samples:", x_test.shape)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-d
atasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 3s 0us/step
Training samples: (60000, 28, 28)
Test samples: (10000, 28, 28)
```

Preprocess Data

In [4]:
```
# Normalize pixel values
x_train = x_train / 255.0
```

```
x_test = x_test / 255.0

# Flatten images (28x28 -> 784)
x_train = x_train.reshape(-1, 784)
x_test = x_test.reshape(-1, 784)

print("Flattened shape:", x_train.shape)
```

```
Flattened shape: (60000, 784)
```

Build Neural Network Model

In [5]:
```
model = keras.Sequential([
    layers.Dense(128, activation='relu', input_shape=(784,)),
    layers.Dropout(0.2),
    layers.Dense(10, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()
```

```
/home/aayushdai/Desktop/Study Materials/AD/venv/lib/python3.12/site-packag
es/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_
shape`/`input_dim` argument to a layer. When using Sequential models, pref
er using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
E0000 00:00:1773384802.253687   18160 cuda_platform.cc:52] failed call to
cuInit: INTERNAL: CUDA error: Failed call to cuInit: UNKNOWN ERROR (303)
```

Model: "sequential"

Layer (type)	Output Shape	P
dense (Dense)	(None, 128)	1
dropout (Dropout)	(None, 128)	
dense_1 (Dense)	(None, 10)	

Total params: 101,770 (397.54 KB)

Trainable params: 101,770 (397.54 KB)

Non-trainable params: 0 (0.00 B)

Train Model

In [6]:
```
history = model.fit(
    x_train,
    y_train,
    epochs=10,
    batch_size=32,
```

```
    validation_split=0.1
)
```

```
Epoch 1/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.9109 - loss: 0.3
088 - val_accuracy: 0.9685 - val_loss: 0.1224
Epoch 2/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 9s 6ms/step - accuracy: 0.9565 - loss: 0.14
87 - val_accuracy: 0.9705 - val_loss: 0.1015
Epoch 3/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.9667 - loss: 0.1
097 - val_accuracy: 0.9750 - val_loss: 0.0836
Epoch 4/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 9s 6ms/step - accuracy: 0.9723 - loss: 0.09
00 - val_accuracy: 0.9780 - val_loss: 0.0823
Epoch 5/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9753 - loss: 0.07
76 - val_accuracy: 0.9802 - val_loss: 0.0727
Epoch 6/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9784 - loss: 0.06
84 - val_accuracy: 0.9780 - val_loss: 0.0762
Epoch 7/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9806 - loss: 0.06
00 - val_accuracy: 0.9803 - val_loss: 0.0703
Epoch 8/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9822 - loss: 0.05
40 - val_accuracy: 0.9810 - val_loss: 0.0734
Epoch 9/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9844 - loss: 0.04
88 - val_accuracy: 0.9808 - val_loss: 0.0725
Epoch 10/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9859 - loss: 0.04
25 - val_accuracy: 0.9832 - val_loss: 0.0731
```

Evaluate Model

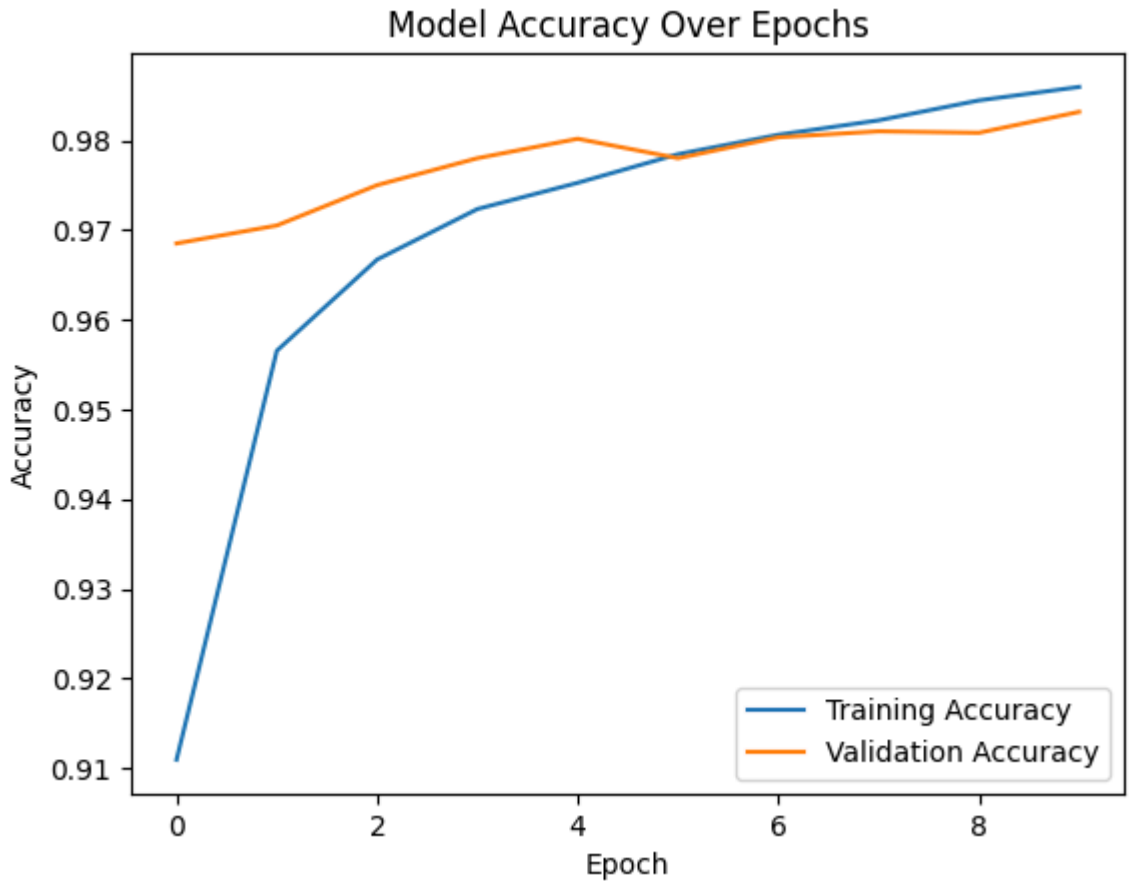
In [7]:
```
test_loss, test_acc = model.evaluate(x_test, y_test)
print("\nTest Accuracy:", test_acc)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9771 - loss: 0.0774

Test Accuracy: 0.9771000146865845
```

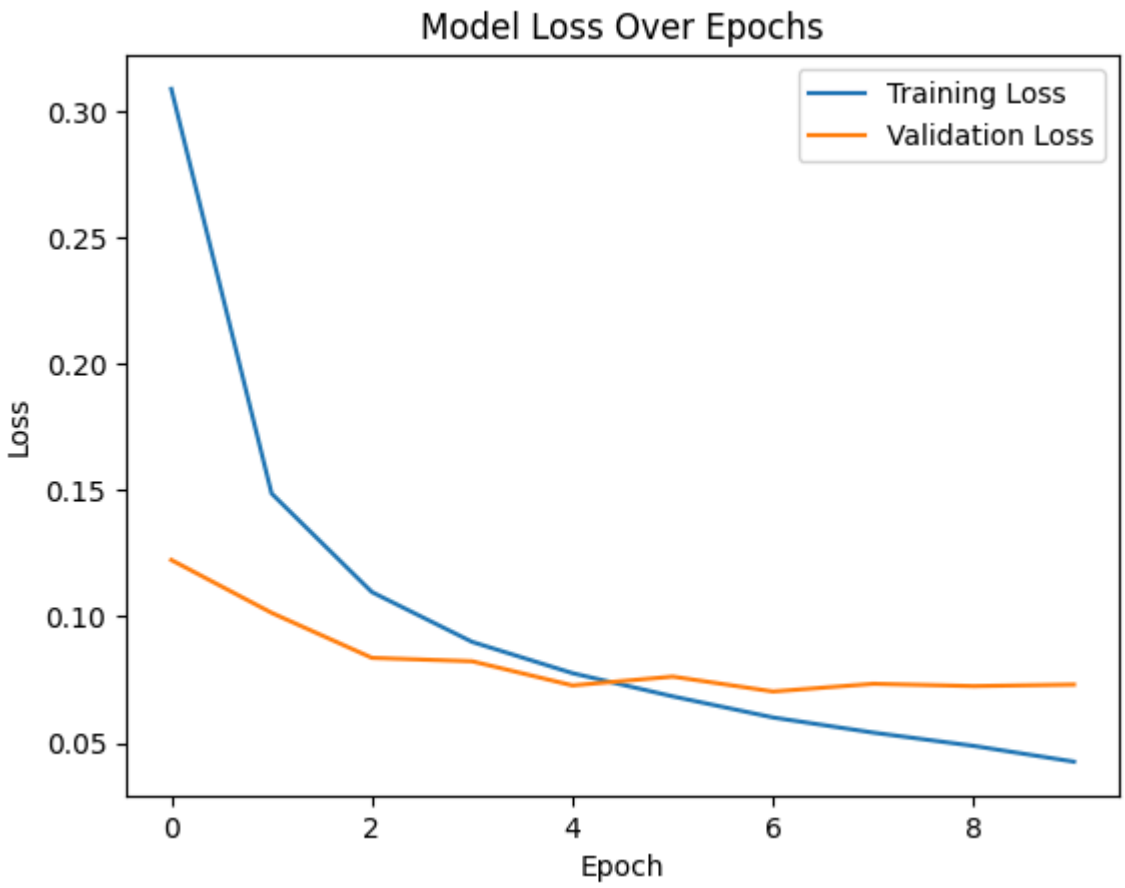
Plot Accuracy Over Epochs

In [8]:
```
plt.figure()
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.savefig('accuracy_plot.png')
plt.show()
```



Plot Loss Over Epochs

In [9]:
```
plt.figure()
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.savefig('loss_plot.png')
plt.show()
```



Show Example Predictions

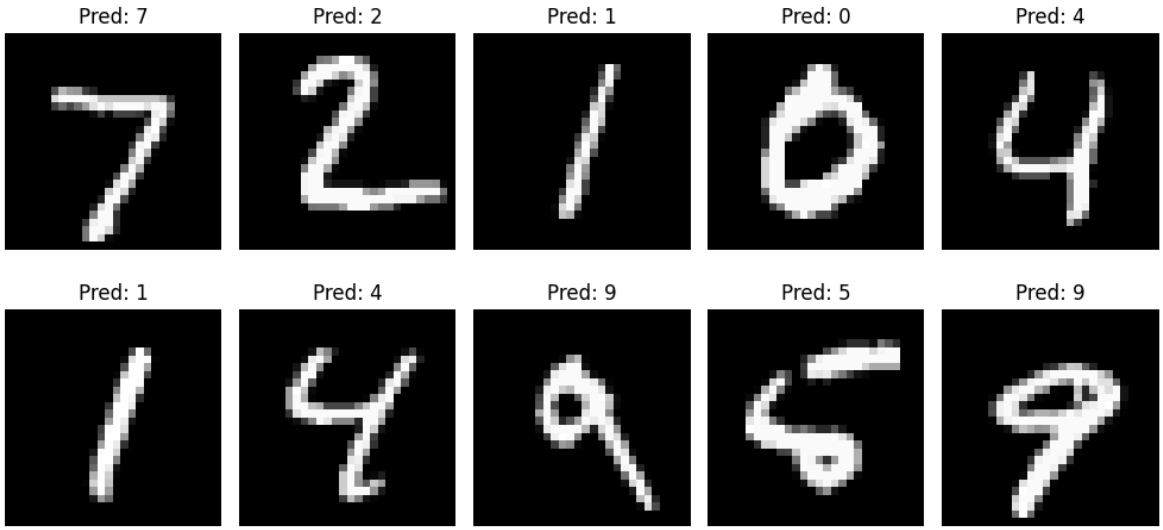
In [10]:
```
predictions = model.predict(x_test)

plt.figure(figsize=(10,5))

for i in range(10):
    plt.subplot(2,5,i+1)
    img = x_test[i].reshape(28,28)
    plt.imshow(img, cmap='gray')
    pred_label = np.argmax(predictions[i])
    plt.title(f"Pred: {pred_label}")
    plt.axis('off')

plt.tight_layout()
plt.savefig('sample_predictions.png')
plt.show()
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
```


Pred: 7
Pred: 2
Pred: 1
Pred: 0
Pred: 4
Pred: 1
Pred: 4
Pred: 9
Pred: 5
Pred: 9